

課題研究 実装計画

グループウェアの研究

目的

現在、十分満足に使用することができない既存のワークスペースソフトを新しく開発することで、開発プロジェクトの効率化・利便性向上を図る。

特に以下の課題解決をはかる。

- ・TODOリストとカレンダーが分離されていることによる不便
- ・既存のワークスペースソフトウェアの無料プラン制限による使いにくさ
- ・会議(リモートも含む)での議事録作成の負担
- ・その他、既存のソフトに存在する機能を実装、開発中、必要に合わせ随時機能を追加
- ・APIを介してweb、android、iosにそれぞれ対応させる

上記のソフトウェアを制作するための第一段階として、課題研究では、グループウェアの基本機能(ベースとなるユーザー管理システム)の実装を通してグループウェアの仕組みについて研究する。他の機能は拡張として実装。

概要

システム構成

- ・APIサーバー
ユーザー管理、チーム管理、チャンネル管理、タスク管理、カレンダー管理
クライアントからのREST APIリクエストを処理し、データベースとの連携を行う
- ・リアルタイムサーバー
WebSocketを用いたリアルタイム通信
リアルタイムチャット、オンライン状態管理、通知送信等を行う。
Redisを介してAPIサーバーと連携し、状態の同期を行う
- ・ボイスサーバー
WebRTCを用いた低遅延音声通信
- ・Redis
APIサーバーとリアルタイムサーバー間のデータ同期およびメッセージ仲介
セッション管理および一時データの高速処理
- ・PostgreSQL
ユーザー情報、チーム情報、メッセージ、タスク、予定等のデータを管理
- ・オブジェクトストレージ
画像・動画・その他ファイルの保存

※単一サーバー内でDocker分離を行う

PWAを介してモバイル端末でもネイティブアプリのように動作させる

想定ユーザー

開発プロジェクトに携わるエンジニア等

計画

本システムは、リアルタイム性、業務向け設計、タスク管理機能を統合した、無料の開発プロジェクト特化型ワークスペースである。

プロセス

4～6月前半：調査・設計

目標

- 技術選定を固める
- 開発環境を統一する
- システム構成を理解する
- 基盤設計を固める

作業内容

- FastAPI / React / PostgreSQL の学習
- Docker の基本理解
- APIサーバー構成の設計
- DB設計
- 画面遷移図作成
- API仕様の整理
- 認証方式の検討
- ディレクトリ構成整理
- PWAの動作確認

成果物

- 要件定義書
- ER図
- API一覧
- 画面設計
- 開発環境テンプレート

6月後半～7月：認証・ユーザー管理実装

目標

ユーザー管理の動作をさせる。

作業内容

- ユーザー登録API
- ログインAPI
- JWT認証
- パスワードハッシュ化
- プロフィール編集
- アイコン画像アップロード
- React側ログイン画面
- Zustandによる認証状態管理

成果物

- ログイン可能なWebアプリ
- DBにユーザー情報保存
- 認証付きAPI

8月: チーム・ロール機能

目標

複数人利用を可能にする。

作業内容

- チーム作成
- 招待機能
- ロール作成
- 権限管理
- チームメンバー一覧
- ロール色表示
- 権限チェックMiddleware

成果物

- チーム管理
- ロールによる権限制御

9月: チャット基盤

目標

リアルタイム機能の基礎を作る。

作業内容

- WebSocket通信
- テキストチャンネル

- メッセージ送信
- メッセージ保存
- Redis導入
- オンライン状態管理
- ファイル送信(余裕があれば)

成果物

- リアルタイムチャット
- メッセージ履歴保存

注意

WebSocketはまずは単純な1チャンネル構成で作る。

10月:タスク・カレンダー機能(余裕があれば)

目標

目指す統合グループウェアにむけて拡張機能を増やす

作業内容

- タスク管理
- ステータス管理
- 優先度管理
- カレンダー表示
- フィルター機能
- ソート機能
- タスクと予定の関連整理

成果物

- TODO管理
- カレンダー画面
- チームごとのタスク共有

11月前半~:UI改善・統合

目標

完成度のアップ

作業内容

- UI整理
- モバイル対応
- Tailwind調整

- エラー処理改善
- API整理
- Docker統合

成果物

- 一通り動く統合版
- デモ可能状態

仕様技術・開発環境

フロントエンド

1. Web
 - ・HTML
画面構造の定義
 - ・CSS / Tailwind(フレームワーク)
高速なUI構築
 - ・TypeScript / React(フレームワーク)
保守性の高いUI
 - ・Zustand(状態管理ライブラリ)
グローバルな状態管理
2. モバイル
 - ・PWA
単一コードベースでAndroid / iOS両方に対応させ、
ネイティブアプリに近い操作性を実現する

バックエンド

1. API
 - ・Python / FastAPI(フレームワーク)
非同期処理による高速なAPI提供
 - ・WebSocket (通信プロトコル)
リアルタイムチャットおよび通知の実現のため使用
2. リアルタイム通信
 - ・WebRTC
低遅延な音声通信を実現するため使用
 - ・Opus Codec
高音質かつ低ビットレートな音声圧縮を行う
3. データベース
 - ・PostgreSQL
ユーザー、メッセージ、タスク等のデータを管理
 - ・Redis
セッション管理、リアルタイム通知、サーバー間連携および負荷分散

4. 議事録作成

- ・ローカルLLM

会議音声の文字起こし、結果要約による議事録作成

5. 開発・運用環境

- ・Docker

各コンポーネントをコンテナ分離

- ・浜工サーバー

各種機能提供のため、使わせてください。